CSCI 460 Fall 2026

Programming Assignment I

Task I: Data download

Download the **Bank Marketing** (https://archive.ics.uci.edu/dataset/222/bank+marketing) dataset from the UC Irvine Machine Learning repository (https://archive.ics.uci.edu/) and use it.

Download Instructions

- After extracting the zipped data file, use only bank-full.csv
- In bank-full.csv, replace all semi-colon (;) with a comma (,) so you can load it as a comma-separated file in excel. Basically, change the field separator to comma.
- Alternatively, you can read it into Python as a text file by setting field separator to semi-colon (sep = ';').
- The file bank-names.txt contains complete data description for convenience
- Please discard the additional data found in bank-additional.zip and bank.csv

Data Preparation Instructions

The task in this project is to construct a Naïve Bayes classifier from the Bank Marketing dataset you downloaded. Step by step, you will train the model, test it and experiment with different proportions of splitting the data to show best performance using accuracy and F1-score metrics.

Write a Python program to do the following:

- Load the above dataset file into a data frame and call it df
- Map each categorical variables into an integer number starting at 0. This applies to the columns: 'job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome', and the target column 'y' (see the mapping example in the hints below)
- Remove any rows that have missing (NA) values from df
- Set X = all feature columns in the data frame excluding the target column y
- Set y = the target column 'y'

Task II: Train and Test a Naive Bayes Model

To build your classification model:

- Split the dataset in X and y randomly into 70% for training and 30% for testing using **stratified** random sampling on the target variable (so that both training and testing data have the same proportions of the target class as the original dataset).
- Build a Gaussian classifier by calling: GaussianNB()
- Fit (or train) the model to the training data.
- Obtain model predictions on test data.

To evaluate the classification performance of your model, use the following Python code: (the code below assumes that y_pred and y_test contain predicted and original classes respectively.)

```
# import performance metric libraries
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score,
    classification_report,
)

# measure the performance
accuracy = accuracy_score(y_pred, y_test)
f1 = f1_score(y_pred, y_test, average="weighted")
print("The accuracy of my Naive Bayes Model is:", accuracy)
print("The F1 Score of my Naive Bayes Model is:", f1)
```

Run your program above repeatedly over 10 iterations and report the average and standard deviation of accuracy and F1-scores over the 10 trials. In your report, present a table of Accuracy and F1-score values for each trial and summarize with mean and standard deviation at the end.

Task III: report impact of splitting the data

Run your program 9 times (10 iterations each) and for each data split, use an increasing percentage for training data by an additional 10% starting with 10% as follows:

- iteration 1: use 10% for training and 90% for testing
- iteration 2: use 20% for training and 80% for testing
- iteration 3: use 30% for training and 70% for testing
- ……..
- iteration 8: use 80% for training and 20% for testing
- iteration 9: use 90% for training and 10% for testing

Please report your performance as a table, as well as, a plot for an increasing size of the training data. Write a brief report (1-2 pages) to discuss your experiment design and any observations you can make on how the increased training data impacts the classification performance based on your findings. Please try to explain any unusual behavior of your model by examining the mean and standard deviation of each of the performance metric.

Useful Hints:

- Example: to map the categorical column 'loan' to integer values, you may use the following Python code fragment:

```
d = {"yes":1,"no":0}                    # list integer values for each label in d
df['loan'] = df['loan'].map(d)          # map d on all rows of df['loan']
```

- To learn more details on Naive Bayes classification, read the following link: https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/
- For illustration, a good example on how to construct a Naïve Bayes classifier, read the following link: https://www.geeksforgeeks.org/machine-learning/gaussian-naive-bayes/
- You'll need to import the necessary libraries! Use: sklearn.naive_bayes